

Computação Gráfica e Processamento de Imagem



Professor: EDILSON LIMA

ATIVIDADE PRÁTICA – Modelo de Câmera

Atividade Prática: Design de Interface para um Painel Urbano Interativo

Ferramenta sugerida: Google Colab

1. Polilinhas – Criando Formas Geométricas

Situação-problema:

Um designer precisa gerar o contorno de um carro (como o Fusca) usando apenas segmentos de reta. Ele quer testar se uma polilinha pode aproximar curvas complexas.

Conceito:

Polilinha = sequência de vértices conectados por arestas.

$Polilinha = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$

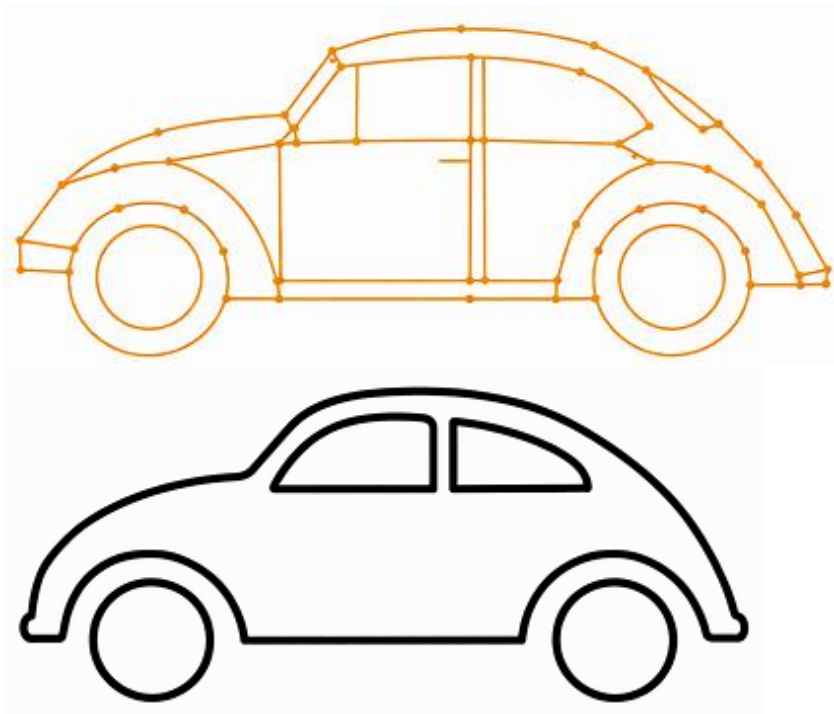
Código (Google Colab):

```
import matplotlib.pyplot as plt

# Vértices aproximando o contorno de um Fusca
fusca = [
    (0,0),(2,1),(4,2),(6,1),(8,0),(6,-1),(4,-2),(2,-1),(0,0)
]

x, y = zip(*fusca)
plt.plot(x, y, '-o', color='blue')
plt.title('Polilinha – Contorno do Fusca')
plt.axis('equal')
plt.show()
```

Desenvolver o código que gere:



Continuação:

Modifique os vértices para criar outras formas geométricas (casa, estrela, coração).

2. Splines – Curvas Suaves a Partir de Pontos

Situação-problema:

O mesmo designer quer suavizar o contorno do Fusca para eliminar os cantos da polilinha.

Conceito:

Spline

cúbica:

$f(t) = at^3 + bt^2 + ct + d$ ou, na forma matricial: $f(t) = G \cdot M \cdot T$ onde (G) é a matriz geométrica (pontos de controle) e (M) é a matriz base.

Código (Google Colab):

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import CubicSpline

x = np.array([0,2,4,6,8])
y = np.array([0,1,2,1,0])

cs = CubicSpline(x, y)
t = np.linspace(0,8,200)
plt.plot(x, y, 'o', label='Pontos de controle')
plt.plot(t, cs(t), '-', label='Spline cúbica')
plt.legend(); plt.axis('equal')
plt.title('Spline cúbica – Curva suave do Fusca')
plt.show()
```

Desafio:

Adicione mais pontos e observe como a spline se ajusta.

3. Curvas de Bézier – Controle Preciso da Forma

Situação-problema:

Um engenheiro quer desenhar o para-lama do Fusca com uma curva suave controlada por quatro pontos.

Equação do slide: $B(t) = (1-t)^3P_0 + 3t(1-t)^2P_1 + 3t^2(1-t)P_2 + t^3P_3$

Código (Google Colab):

```
import numpy as np
import matplotlib.pyplot as plt

def bezier_curve(P, n=100):
    t = np.linspace(0,1,n)
    B = (1-t)**3*P[0] + 3*t*(1-t)**2*P[1] + 3*t**2*(1-t)*P[2] + t**3*P[3]
    return B

P = np.array([[0,0],[2,3],[4,3],[6,0]])
B = bezier_curve(P)

plt.plot(P[:,0], P[:,1], 'o--', label='Pontos de controle')
plt.plot(B[:,0], B[:,1], '-', label='Curva de Bézier')
plt.legend(); plt.axis('equal')
plt.title('Curva de Bézier – Para-lama do Fusca')
plt.show()
```

Desafio:

Mude os pontos de controle para alterar a curvatura.

4. B-Splines – Curvas Complexas com Muitos Pontos

Situação-problema:

Um modelador quer desenhar o teto do Fusca com uma curva mais flexível, usando vários pontos de controle.

Conceito:

B-Spline cúbica com (n) pontos de controle e grau (k).

$$S(t) = \sum_{i=0}^n N_{i,k}(t)P_i$$

Código (Google Colab):

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import make_interp_spline
```

```
x = np.array([0,1,2,3,4,5,6,7,8])
y = np.array([0,1,2,2.5,2.8,2.5,2,1,0])
```

```
spl = make_interp_spline(x, y, k=3)
t = np.linspace(0,8,200)
plt.plot(x, y, 'o', label='Pontos de controle')
plt.plot(t, spl(t), '-', label='B-Spline cúbica')
plt.legend(); plt.axis('equal')
plt.title('B-Spline – Teto do Fusca')
plt.show()
```

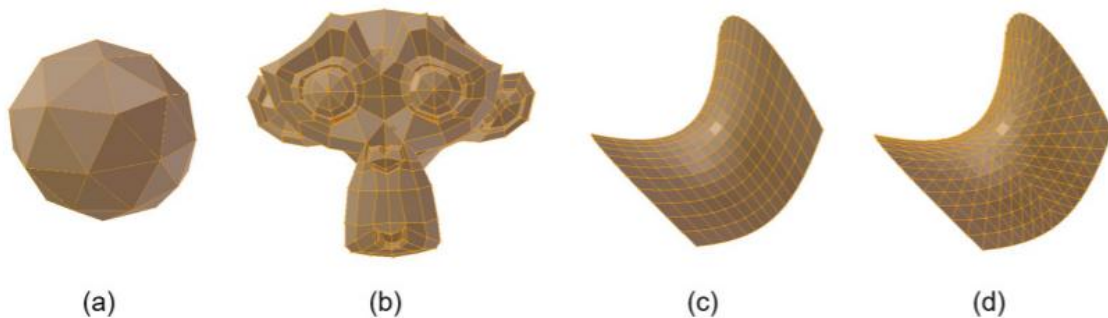
Desafio:

Aumente o número de pontos e veja como a curva se torna mais suave.

5. Poliedros e Malhas – Modelagem 3D (a, b, c, d)

Situação-problema:

Um artista 3D quer comparar diferentes representações de malhas: (a) esfera poligonal, (b) macaco (Suzanne), (c) superfície quadrilateral, (d) superfície triangular.



Conceito:

Malha = conjunto de vértices + lista de triângulos.

malha = (vértices, triângulos)

Código (Google Colab):

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# Exemplo simples de malha triangular
vertices = np.array([
    [0,0,0],[1,0,0],[1,1,0],[0,1,0],[0.5,0.5,1]
])
triangulos = [
    [0,1,4],[1,2,4],[2,3,4],[3,0,4],[0,1,2],[0,2,3]
]

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
for tri in triangulos:
    pts = vertices[tri]
    ax.add_collection3d(Poly3DCollection([pts], alpha=0.5, edgecolor='k'))
ax.scatter(vertices[:,0], vertices[:,1], vertices[:,2], color='red')
ax.set_title('Malha triangular – Exemplo (a-d)')
plt.show()
```

Desafio:

Crie versões (a), (b), (c), (d) variando o tipo de polígonos (triângulos ou quadriláteros) e a densidade da malha.

6. Projeto Integrador – “Curvas do Fusca”**Situação-problema:**

Combine todos os conceitos para modelar o contorno 2D do Fusca com:

- Polilinhas para o corpo;
- Splines para suavizar;
- Bézier para detalhes (para-lama, janelas);
- B-Splines para o teto.

Objetivo:

Gerar uma imagem final do Fusca estilizado com curvas suaves e proporções realistas.